

Lesson 7: Styling with Tailwind CSS and Component Design

Mission Brief

Your portfolio site works, but it's unstyled — no colors, no spacing, no visual identity befitting a Lumexa mission control commander. Hand-writing a separate CSS file for every component gets slow and hard to keep consistent across a big site. Today you learn **Tailwind CSS**, a utility-first framework that lets you style components directly in your JSX with small, composable classes, and you use it to give your Next.js portfolio a real design.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain the difference between utility-first CSS and traditional hand-written CSS
- Style layout, spacing, color, and typography using Tailwind utility classes
- Build responsive designs using Tailwind's breakpoint prefixes
- Apply conditional/dynamic classes based on component state
- Extract repeated class groups into reusable components

What You'll Build

A fully styled `ProjectCard` component and a responsive page layout, using only Tailwind utility classes — no separate `.css` file needed for these components.

Prerequisites

- Comfortable with CSS fundamentals — colors, spacing, flexbox (Course 07)
- Comfortable building and composing React components (Lessons 1-3)
- A Next.js project with Tailwind already configured (as in Project 09's scaffold)

Key Concepts

- Utility-first CSS vs. traditional CSS files
- Common Tailwind utilities: spacing, color, typography, flex/grid
- Responsive prefixes (`sm:` , `md:` , `lg:`)
- Hover/focus state utilities (`hover:` , `focus:`)
- Conditional classes based on props/state
- Extracting a styled component instead of repeating long class strings

Concept Explanation

Traditional CSS means writing rules in a separate file and referencing them by class name: `.card { padding: 1rem; border-radius: 0.5rem; }`. Tailwind flips this around with **utility classes** — small, single-purpose classes you compose directly in your markup, so `padding: 1rem` becomes `p-4` and `border-radius: 0.5rem` becomes `rounded-lg`, applied right on the element:

```
function ProjectCard({ project }) {
  return (
    <div className="rounded-lg border border-slate-700 bg-slate-800 p-4 shadow-md">
      <h3 className="text-lg font-bold text-white">{project.title}</h3>
      <p className="mt-2 text-sm text-slate-300">{project.description}</p>
    </div>
  );
}
```

Each class does one thing: `p-4` is padding, `rounded-lg` is a border radius, `bg-slate-800` is a background color, `text-white` is text color, `mt-2` is a top margin. Reading a list of these classes tells you exactly what's visually happening, without needing to jump to a separate stylesheet — this "locality of behavior" is the main argument for utility-first CSS, and it scales well once you're used to the class names (which follow very consistent patterns: `text-*`, `bg-*`, `p-*` / `m-*` for padding/margin, `w-*` / `h-*` for size).

Layout utilities map directly to concepts you already know from Course 07's Flexbox/Grid lessons:

```
<div className="flex flex-col gap-4 md:flex-row md:flex-wrap">
  {projects.map((p) => <ProjectCard key={p.slug} project={p} />)}
</div>
```

`flex flex-col gap-4` stacks cards vertically with spacing between them by default (mobile-first). The `md:flex-row md:flex-wrap` prefix means "starting at the `md` breakpoint (medium screens and up), switch to a wrapping horizontal row instead." Tailwind's responsive prefixes — `sm:`, `md:`, `lg:`, `xl:` — always apply "at this width and above," which is why you write your base (mobile) styles with no prefix, then add prefixed overrides for larger screens — the same mobile-first philosophy from Course 07, Lesson 6.

Interactive states use their own prefixes too:

```
<button className="rounded bg-indigo-600 px-4 py-2 text-white hover:bg-indigo-500
focus:outline-none focus:ring-2 focus:ring-indigo-400">
  View Project
</button>
```

`hover:bg-indigo-500` only applies on hover, `focus:ring-2` only when the element is focused (e.g. via keyboard Tab) — no separate CSS rule needed.

You can also make classes conditional based on props or state, exactly like any other JSX expression:

```
function StatusBadge({ status }) {
  const isLive = status === "live";
  return (
    <span
      className={`rounded-full px-3 py-1 text-xs font-semibold ${
        isLive ? "bg-emerald-600 text-white" : "bg-slate-600 text-slate-200"
      }`}
    >
      {isLive ? "🟢 Live" : "🚧 In Progress"}
    </span>
  );
}
```

Here a template literal builds up the final `className` string, mixing always-on classes with a ternary that swaps in different color classes depending on the `status` prop.

Finally, when the same long class string starts repeating across your app (say, every button on the site), extract it into a small reusable component instead of copy-pasting the classes everywhere:

```
function Button({ children, ...props }) {
  return (
    <button
      className="rounded bg-indigo-600 px-4 py-2 text-white hover:bg-indigo-500"
      {...props}
    >
      {children}
    </button>
  );
}
```

Now `<Button onClick={...}>Send</Button>` gets the styling automatically, and changing the look of every button on the site means editing one file.

Guided Walkthrough

1. In your Next.js project, style a `ProjectCard` component with the classes shown above (border, background, padding, rounded corners, text colors).
2. Build a responsive grid of cards: `flex flex-col gap-4 md:grid md:grid-cols-2 lg:grid-cols-3`, and resize your browser window to confirm the layout changes from a single column to two, then three, at the right breakpoints.
3. Add `hover:` and `focus:` utility classes to a button and confirm the visual change on mouse-hover and on keyboard Tab-focus.
4. Build the `StatusBadge` component with conditional classes driven by a `status` prop, and render it with both `"live"` and some other value to confirm both visual states work.

- Find two components using the same long `className` string for buttons, and extract a shared `Button` component that both now use instead.

Common Bugs & Debugging Tips

- If Tailwind classes don't seem to apply at all, confirm `globals.css` includes the Tailwind directives/import and that the project's `tailwind.config` (or Tailwind v4 setup) actually scans your component files — a misconfigured `content` /scan path silently produces unstyled output.
- A responsive class like `md:flex-row` doing nothing usually means you forgot the mobile-first base class (`flex-col`) it's meant to override, or you're testing at a browser width narrower than the `md` breakpoint (Tailwind's default `md` is 768px).
- Extremely long `className` strings that wrap awkwardly are normal in Tailwind — resist the urge to abbreviate; instead extract a component (like `Button` above) once a class string repeats three or more times.
- A missing space between concatenated classes in a template literal (``rounded-full${isLive ? ... }`` without a space) silently merges two class names into one invalid string — always double check spacing in dynamic class strings.

Check Your Understanding

- What is the core difference between utility-first CSS and traditional CSS files?
- What does the `md:` prefix mean on a Tailwind class, and which direction does it apply?
- How do you apply a style only on hover, without writing separate CSS?
- Why extract a `Button` component instead of repeating a long class string on every button?
- What mobile-first assumption does Tailwind make about unprefixed classes?

Mini Quiz + Answer Key

- Q1.** What does the Tailwind class `p-4` do? A) Sets font size B) Adds padding C) Sets position D) Adds a border
- Q2.** What does `md:flex-row` mean? A) Always a row layout B) A row layout only below the `md` breakpoint C) A row layout starting at the `md` breakpoint and up D) It disables flex entirely
- Q3.** How do you style an element only while it's being hovered, in Tailwind? A) Write a separate `:hover` CSS rule B) Use a class prefixed with `hover:`, e.g. `hover:bg-indigo-500` C) It's not possible in Tailwind D) Use `onMouseOver` only
- Q4.** Why extract repeated long class strings into a shared component? A) Tailwind requires it B) To keep styling consistent and avoid repeating the same edit in many places C) It makes the app load faster D) It's not recommended, always repeat classes

Answer Key: 1-B, 2-C, 3-B, 4-B

Lesson Recap

You learned Tailwind's utility-first approach to styling directly in JSX, how responsive (`md:`) and state (`hover:` , `focus:`) prefixes work, how to build conditional classes from props/state, and when to extract a shared styled component instead of repeating class strings.

Homework

Fully restyle your `NavBar` and `Footer` components using Tailwind utilities to match a consistent color theme (pick 2-3 colors and reuse them throughout), make the `NavBar` responsive (a horizontal row on desktop, and reasonably usable on a narrow phone width), and extract a shared `Button` component used in at least two different places.

Portfolio Project Connection

Project 09's entire visual design — cards, navigation, buttons, responsive grid of projects — is built with exactly the Tailwind utility patterns from this lesson.